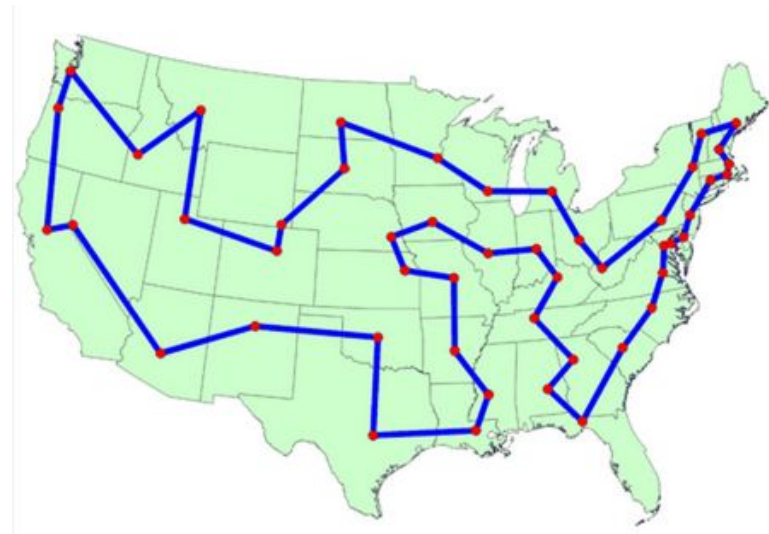




Lecture 13

Introduction to Asymptotic Analysis

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Goal: Measuring Code Efficiency

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- Intuitive Runtime Characterization

Order of Growth

- Defining Order of Growth
- Counting Operations (Tedious Approach)
- Ignoring Low Order Terms
- Why Scaling Matters

Asymptotic Notation

- Big Theta (a.k.a. Order of Growth)
- Big O

An engineer will do for a dime what any fool will do for a dollar.

Efficiency comes in two flavors:

- Programming cost (course to date. Will also revisit later).
 - How long does it take to develop your programs?
 - How easy is it to read, modify, and maintain your code?
 - More important than you might think!
 - Majority of cost is in maintenance, not development!
- Execution cost (from today until end of course).
 - How much time does your program take to execute?
 - How much memory does your program require?

Intuitive Runtime Characterization

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- **Intuitive Runtime Characterization**

Order of Growth

- Defining Order of Growth
- Counting Operations (Tedious Approach)
- Ignoring Low Order Terms
- Why Scaling Matters

Asymptotic Notation

- Big Theta (a.k.a. Order of Growth)
- Big O

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

Attendance:



If someone asked you how long `countEvens` takes to run, what would you tell them?

Example of Algorithm Cost

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

N: length of numbers

If someone asked you how long countEvens takes to run, what would you tell them?

- Runtime depends how fast the computer is
- Runtime depends on the size of the array
 - Do some work for each item in the array
- Runtime “grows like N”
 - N: size of the input

Defining Order of Growth

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- Intuitive Runtime Characterization

Order of Growth

- **Defining Order of Growth**
- Counting Operations (Tedious Approach)
- Ignoring Low Order Terms
- Why Scaling Matters

Asymptotic Notation

- Big Theta (a.k.a. Order of Growth)
- Big O

Order of Growth

As N (size of input) grows, what happens to the runtime of the algorithm?

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

What about initializing evens, and printing the result?

Order of growth: N

If N doubles, runtime doubles

Order of Growth

As N (size of input) grows, what happens to the runtime of the algorithm?

- Focus on behavior as N gets large → ignore low order terms

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

Time to initialize evens and print the result becomes negligible as N grows

Order of growth: N

If N doubles, runtime doubles

Order of Growth

As N (size of input) grows, what happens to the runtime of the algorithm?

- Focus on behavior as N gets large → ignore low order terms
- Ignore multiplicative constants
 - $8N$ grows in the same way that N does

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

Time to initialize evens and print the result becomes negligible as N grows

Order of growth: N

If N doubles, runtime doubles

Counting Operations (Tedious Approach)

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- Intuitive Runtime Characterization

Order of Growth

- Defining Order of Growth
- **Counting Operations (Tedious Approach)**
- More Examples
- Why Scaling Matters

Asymptotic Notation

- Big Theta (a.k.a. Order of Growth)
- Big O

Counting Operations To Determine Order of Growth

Count possible operations in terms of input array size N

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

operation	count
evens = 0	1
i = 0	1
Check i < numbers...	N + 1
i++	N
% 2	N
== 0	N
Evens += 1	0 to N
print	1

Counting Operations To Determine Order of Growth

Count possible operations in terms of input array size N

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

Order of growth: As N grows, what happens to the runtime of the algorithm?

- Ignore low-order terms
- Ignore multiplicative constants

operation	count
evens = 0	1
i = 0	1
i < numbers.length	N + 1
i++	N
% 2	N
equals (==)	N
evens += 1	0 to N
IO.println	1

Counting Operations To Determine Order of Growth

Count possible operations in terms of input array size N

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

Order of growth: As N grows, what happens to the runtime of the algorithm?

- Ignore low-order terms
- Ignore multiplicative constants

operation	count
evens = 0	1
i = 0	1
i < numbers.length	N + 1
i++	N
% 2	N
equals (==)	N
evens += 1	0 to N
IO.println	1

Counting Operations To Determine Order of Growth

Count possible operations in terms of input array size N

```
public static void countEvens(int[] numbers)
{
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

operation	count
evens = 0	1
i = 0	1
i < numbers.length	N + 1
i++	N
% 2	N
equals (==)	N
evens += 1	0 to N
IO.println	1

Order of growth: As N grows, what happens to the runtime of the algorithm?

All reasonable
cost models

- Ignore low-order terms
- Ignore multiplicative constants
- Choose representative operation as **cost model**

Cost Model Recap

Goal: **Order of growth**: As N grows, what happens to the runtime of the algorithm?

Choose representative operation as **cost model**;
use count of that operation as order of growth

Ignore low-order terms

Ignore multiplicative constants

Choose `==` as cost model:

Order of growth is N

All reasonable
cost models

operation	count
<code>evens = 0</code>	1
<code>i = 0</code>	1
<code>i < numbers.length</code>	$N + 1$
<code>i++</code>	N
<code>% 2</code>	N
<code>equals (==)</code>	N
<code>evens += 1</code>	0 to N
<code>IO.println</code>	1

More Examples

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- Intuitive Runtime Characterization

Order of Growth

- Defining Order of Growth
- Counting Operations (Tedious Approach)
- **More Examples**
- Why Scaling Matters

Asymptotic Notation

- Big Theta (a.k.a. Order of Growth)
- Big O

```
public static void count1(int[] numbers) {
    int evens = 0;
    int odds = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        } else {
            odds += 1;
        }
    }
    IO.println("Number of evens: " + evens);
    IO.println("Number of odds: " + odds);
}
```



```
public static void count2(int[] numbers) {
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    int odds = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 != 0) {
            odds += 1;
        }
    }
    IO.println("Number of evens: " + evens);
    IO.println("Number of odds: " + odds);
}
```

What's the order growth of count1 and count2?

- Focus on behavior as N gets large → ignore low order terms
- Ignore multiplicative constants

Order of Growth: Counting Evens and Odds

```
public static void count1(int[] numbers) {  
    int evens = 0;  
    int odds = 0;  
    for (int i = 0; i < numbers.length; i++) {  
        if (numbers[i] % 2 == 0) {  
            evens += 1;  
        } else {  
            odds += 1;  
        }  
    }  
}
```

```
IO.println  
IO.println
```

Order of growth: N

If N doubles, runtime doubles

```
public static void count2(int[] numbers) {  
    int evens = 0;  
    for (int i = 0; i < numbers.length; i++) {  
        if (numbers[i] % 2 == 0) {  
            evens += 1;  
        }  
    }  
    int odds = 0;  
    for (int i = 0; i < numbers.length; i++) {  
        if (numbers[i] % 2 != 0) {  
            odds += 1;  
        }  
    }  
}
```

```
IO.println  
IO.println
```

Order of growth: N

If N doubles, runtime doubles

What's the order growth of count1 and count2?

- Focus on behavior as N gets large → ignore low order terms
- Ignore multiplicative constants

Order of Growth: Counting Duplicates

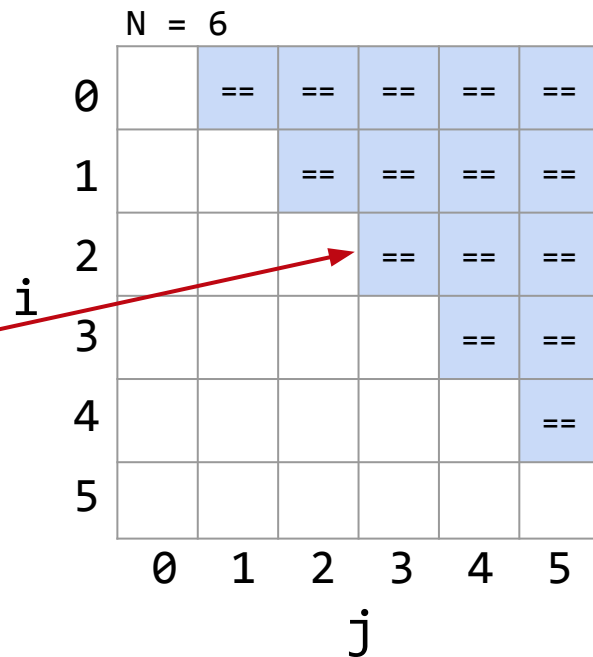
```
public static void countDuplicates(int[] a) {  
    int duplicates = 0;  
    for (int i = 0; i < a.length; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (a[i] == a[j]) {  
                duplicates += 1;  
            }  
        }  
    }  
    IO.println("Duplicates: " + duplicates);  
}
```

Intuitive approach: how many pairs do we check?

Cost model: ==

What's the order growth of countDuplicates?

- Area of right triangle: $\sim N^2 / 2$
- Order of growth is N^2



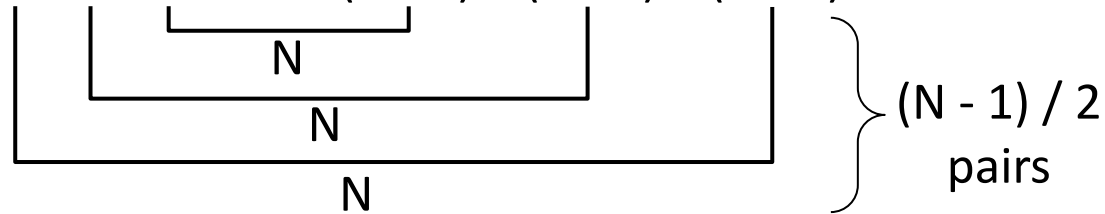
Order of Growth: Counting Duplicates

```
public static void countDuplicates(int[] a) {  
    int duplicates = 0;  
    for (int i = 0; i < a.length; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (a[i] == a[j]) {  
                duplicates += 1;  
            }  
        }  
    }  
    IO.println("Duplicates: " + duplicates);  
}
```

Intuitive approach: how many pairs do we check?

Cost model: ==

$$1 + 2 + 3 + \dots + (N - 3) + (N - 2) + (N - 1)$$



$$= N(N - 1)/2$$

Order of growth is N^2

$N = 6$

0		==	==	==	==	==
1			==	==	==	==
2				==	==	==
3					==	==
4						==
5						
i	0	1	2	3	4	5
	j					

```
public static void countZerps(int[] a) {  
    int zerps = 0;  
    for (int i = 0; i < a.length; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (isZerp(a[i], a[j])) {  
                zerps += 1;  
            }  
        }  
    }  
    IO.println("Zerps: " + zerps);  
}
```

What's the order growth of count?

- Focus on behavior as N gets large
→ ignore low order terms
- Ignore multiplicative constants



```
public static void countZerps(int[] a) {  
    int zerps = 0;  
    for (int i = 0; i < a.length; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (isZerp(a[i], a[j])) {  
                zerps += 1;  
            }  
        }  
    }  
    IO.println("Zerps: " + zerps);  
}
```

What's the order growth of count?

- Focus on behavior as N gets large
→ ignore low order terms
- Ignore multiplicative constants

Order of growth is N^2

(assuming that isZerp's runtime depends only on its two parameters, and not on N)

```
public static void countEvens(int[] numbers) {  
    int evens = 0;  
    for (int i = 0; i < numbers.length; i++) {  
        if (numbers[i] % 2 == 0) {  
            evens += 1;  
        }  
    }  
    IO.println("Number of evens: " + evens);  
}
```

Order of growth: N

```
public static void countDuplicates(int[] a) {  
    int duplicates = 0;  
    for (int i = 0; i < a.length; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (a[i] == a[j]) {  
                duplicates += 1;  
            }  
        }  
    }  
    IO.println("Duplicates: " + duplicates);  
}
```

Order of growth: N^2

```
public static void count(int[] numbers) {  
    countEvens(numbers);  
    countDuplicates(numbers);  
}
```

What's the order growth of count?

- Focus on behavior as N gets large
→ ignore low order terms
- Ignore multiplicative constants




```
public static void countEvens(int[] numbers) {
    int evens = 0;
    for (int i = 0; i < numbers.length; i++) {
        if (numbers[i] % 2 == 0) {
            evens += 1;
        }
    }
    IO.println("Number of evens: " + evens);
}
```

Order of growth: N

```
public static void countDuplicates(int[] a) {
    int duplicates = 0;
    for (int i = 0; i < a.length; i++) {
        for (int j = i + 1; j < a.length; j++) {
            if (a[i] == a[j]) {
                duplicates += 1;
            }
        }
    }
    IO.println("Duplicates: " + duplicates);
}
```

Order of growth: N^2

```
public static void count(int[] numbers) {
    countEvens(numbers);
    countDuplicates(numbers);
}
```

What's the order growth of count?

- Focus on behavior as N gets large
→ ignore low order terms
- Ignore multiplicative constants

Drop this low order term!
Doesn't matter as N gets big

Order of growth is N^2

Why Scaling Matters

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- Intuitive Runtime Characterization

Order of Growth

- Defining Order of Growth
- Counting Operations (Tedious Approach)
- More Examples
- **Why Scaling Matters**

Asymptotic Notation

- Big Theta (a.k.a. Order of Growth)
- Big O

In most cases, we care only about asymptotic behavior, i.e. what happens for very large N .

- Simulation of billions of interacting particles.
- Social network with billions of users.
- Logging of billions of transactions.
- Encoding of billions of bytes of video data.

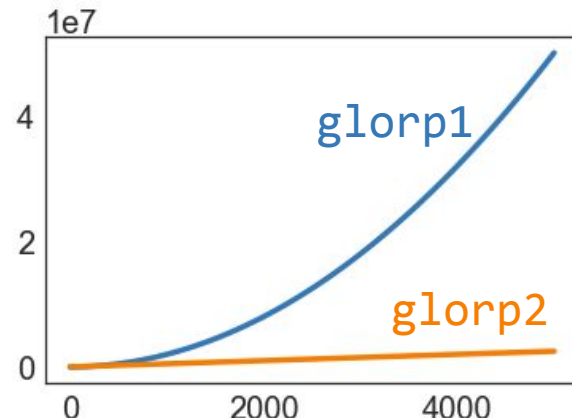
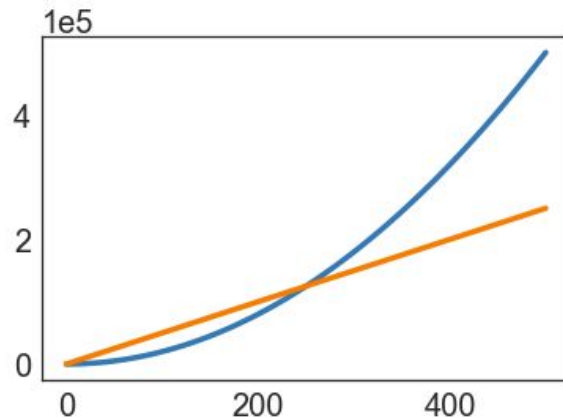
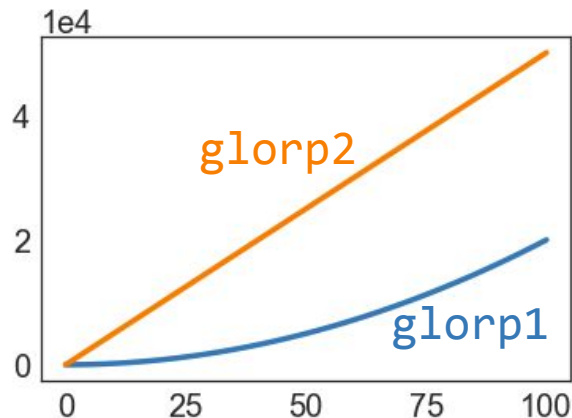
Algorithms which scale well (e.g. look like lines) have better asymptotic runtime behavior than algorithms that scale relatively poorly (e.g. look like parabolas).

Parabolas vs. Lines

Suppose we have two algorithms that glorpify a collection of N items.

- glorp1 takes $2N^2$ operations.
- glorp2 takes $500N$ operations.

For small N , glorp1 might be faster, but as dataset size grows, the parabolic algorithm is going to fall farther and farther behind (in time it takes to complete).



Algorithm runtimes, for different **orders of growth**

- Effect is dramatic! Often determines whether a problem can be solved at all.

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

Table 2.1 The running times (rounded up) of different algorithms on inputs of increasing size, for a processor performing a million high-level instructions per second. In cases where the running time exceeds 10^{25} years, we simply record the algorithm as taking a very long time.

(from Algorithm Design: Tardos, Kleinberg)

Big Theta (a.k.a. Order of Growth)

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- Intuitive Runtime Characterization

Order of Growth

- Defining Order of Growth
- Counting Operations (Tedious Approach)
- More Examples
- Why Scaling Matters

Asymptotic Notation

- **Big Theta (a.k.a. Order of Growth)**
- Big O

Formalizing Order of Growth

Given a function $Q(N)$, we can apply our two simplifications:

- Ignore low orders terms
- Ignore multiplicative constants

To yield the order of growth of $Q(N)$.

- Example: $Q(N) = 3N^3 + N^2$
- Order of growth: N^3

Let's finish out this lecture by moving to a more formal notation called Big-Theta.

- The math might seem daunting at first.
- ... but the idea is exactly the same! Using “Big-Theta” instead of “order of growth” does not change the way we analyze code at all.

Order of Growth Exercise

What's the order of growth of each function below?

- Informally, what is the “shape” of each function for very large N ?

function	order of growth
$N^3 + 3N^4$	
$1/N + N^3$	
$1/N + 5$	
$N e^N + N$	
$40 \sin(N) + 4N^2$	

Range is $[-1, 1]$

Order of Growth Exercise

What's the order of growth of each function below?

- Informally, what is the “shape” of each function for very large N ?

function	order of growth
$N^3 + 3N^4$	N^4
$1/N + N^3$	N^3
$1/N + 5$	1
$N^e + N$	N^e
$40 \sin(N) + 4N^2$	N^2

Suppose we have a function $R(N)$ with order of growth $f(N)$.

- In “Big-Theta” notation we write this as $R(N) \in \Theta(f(N))$.
- Examples:
 - $N^3 + 3N^4 \in \Theta(N^4)$
 - $1/N + N^3 \in \Theta(N^3)$
 - $1/N + 5 \in \Theta(1)$
 - $Ne^N + N \in \Theta(Ne^N)$
 - $40 \sin(N) + 4N^2 \in \Theta(N^2)$

function $R(N)$	order of growth
$N^3 + 3N^4$	N^4
$1/N + N^3$	N^3
$1/N + 5$	1
$Ne^N + N$	Ne^N
$40 \sin(N) + 4N^2$	N^2

Big-Theta: Formal Definition

The function we care about

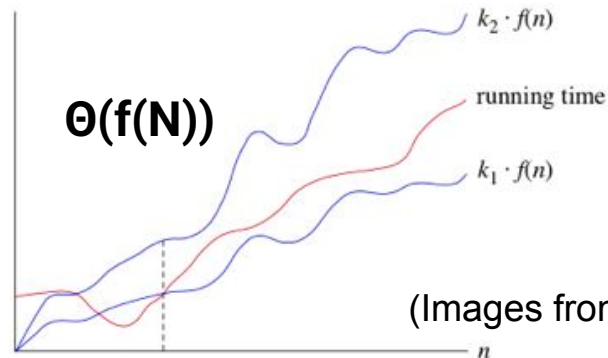
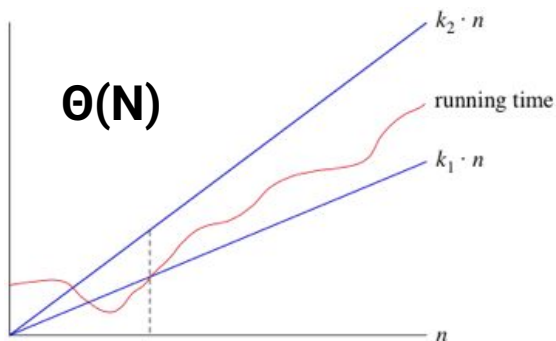
$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

The function we care about

for all values of N greater than some N_0 . i.e. very large N



(Images from Khan Academy)

Big-Theta: Formal Definition

The function we care about


$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

The function we care about

for all values of N greater than some N_0 .  i.e. very large N

Example:

$$3 \cdot N^2 \leq 40 \sin(N) + 4N^2 \leq 5 \cdot N^2$$

$$40 \sin(N) + 4N^2 \in \Theta(N^2)$$

Big-Theta: Formal Definition

The function we care about

$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

The function we care about

for all values of N greater than some N_0 . ← i.e. very large N

$$c_2 \cdot N^2 \leq \begin{array}{l} \text{Runtime of our countEvens and} \\ \text{countDuplicates example:} \\ c_1 \cdot N + c_2 \cdot N^2 \end{array} \leq (c_1 + c_2) \cdot N^2$$

Order of growth $\in \Theta(N^2)$

Suppose $R(N) = (4N^2 + 3N \ln(N))/2$.

- Find a simple $f(N)$ and corresponding k_1 and k_2 .

$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .

← i.e. very large N



Big-Theta Challenge ([Visualization](#)):

Suppose $R(N) = (4N^2 + 3N \ln(N))/2$.

- $f(N) = N^2$
- $k_1 = 1$
- $k_2 = 3$

$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .

 i.e. very large N

Tying Order of Growth to Operation Counts

Consider the algorithm below. What do you expect will be the **order of growth** of the runtime for the algorithm?

operation	count
less than (<)	$100N^2 + 3N$
greater than (>)	$2N^3 + 1$
and (&&)	5,000

Order of growth $\in \Theta N^3$

Argument:

- Suppose < takes α nanoseconds, > takes β nanoseconds, and && takes γ nanoseconds.
- Total time is $\alpha(100N^2 + 3N) + \beta(2N^3 + 1) + 5000\gamma$ nanoseconds.
- For very large N , the $2\beta N^3$ term is much larger than the others **regardless of the values of α , β , and γ .**

Extremely important point.
Make sure you understand it!

Big-Theta and Runtime Analysis

Using Big-Theta doesn't change anything about runtime analysis (no need to find k_1 or k_2 or anything like that).

- The only difference is that we use the Θ symbol anywhere we would have said "order of growth".

```
public static void countDuplicates(int[] a) {  
    int duplicates = 0;  
    for (int i = 0; i < a.length; i++) {  
        for (int j = i + 1; j < a.length; j++) {  
            if (a[i] == a[j]) {  
                duplicates += 1;  
            }  
        }  
    }  
    IO.println("Duplicates: " + duplicates);  
}
```

Order of growth is N^2

Runtime: $\Theta(N^2)$

Big O (Time Permitting)

Lecture 12, CS61B, Spring 2026

Goal: Measuring Code Efficiency

- Intuitive Runtime Characterization

Order of Growth

- Defining Order of Growth
- Counting Operations (Tedious Approach)
- More Examples
- Why Scaling Matters

Asymptotic Notation

- Big Theta (a.k.a. Order of Growth)
- **Big O**

We used Big Theta to describe the order of growth of a function.

function $R(N)$	order of growth
$N^3 + 3N^4$	$\Theta(N^4)$
$1/N + N^3$	$\Theta(N^3)$
$1/N + 5$	$\Theta(1)$
$N^N + N$	$\Theta(N^N)$
$40 \sin(N) + 4N^2$	$\Theta(N^2)$

We also used Big Theta to describe the rate of growth of the runtime of a piece of code.

Whereas Big Theta can informally be thought of as something like “equals”, Big O can be thought of as “less than or equal”.

Example, the following are all true:

- $N^3 + 3N^4 \in \Theta(N^4)$
- $N^3 + 3N^4 \in O(N^4)$
- $N^3 + 3N^4 \in O(N^6)$
- $N^3 + 3N^4 \in O(N!)$
- $N^3 + 3N^4 \in O(N^{N!})$

$$R(N) \in \Theta(f(N))$$

means there exist positive constants k_1 and k_2 such that:

$$k_1 \cdot f(N) \leq R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .

 i.e. very large N

Example: $40 \sin(N) + 4N^2 \in \Theta(N^2)$

- $R(N) = 40 \sin(N) + 4N^2$
- $f(N) = N^2$
- $k_1 = 3$
- $k_2 = 5$

$$R(N) \in O(f(N))$$

means there exists positive constants k_2 such that:

$$R(N) \leq k_2 \cdot f(N)$$

for all values of N greater than some N_0 .

 i.e. very large N

Example: $40 \sin(N) + 4N^2 \in O(N^4)$

- $R(N) = 40 \sin(N) + 4N^2$
- $f(N) = N^4$
- $k_2 = 1$

Big Theta vs. Big O

We will see why big O is practically useful in the upcoming Disjoint Sets lecture.

	Informal meaning:	Family	Family Members
Big Theta $\Theta(f(N))$	Order of growth is $f(N)$.	$\Theta(N^2)$	$N^2/2$ $2N^2$ $N^2 + 38N + N$
Big O $O(f(N))$	Order of growth is less than or equal to $f(N)$.	$O(N^2)$	$N^2/2$ $2N^2$ $\lg(N)$

Given a code snippet, we can express its runtime as a function $R(N)$, where N is some property of the input of the function (often the size of the input).

Rather than finding $R(N)$ exactly, we instead usually only care about the order of growth of $R(N)$.

One approach (not universal):

- Choose a representative operation, and let $C(N)$ be the count of how many times that operation occurs as a function of N .
- Determine order of growth $f(N)$ for $C(N)$, i.e. $C(N) \in \Theta(f(N))$
 - Often (but not always) we consider the worst case count.
- Can use O as an alternative for Θ . O is used for upper bounds.

TSP problem solution, title slide:

http://support.sas.com/documentation/cdl/en/ornoaug/65289/HTML/default/viewer.htm#ornoaug_optnet_examples07.htm#ornoaug.optnet.map002g

Table of runtimes for various orders of growth: Kleinberg & Tardos, Algorithm Design.

Cost model terminology: Algorithms 4th edition by Robert Sedgewick and Kevin Wayne

Charts showing big theta bounds: Khan Academy